```
# -*- coding: utf-8 -*-
# ---------------------------------------------------------------
# Project Name: Uber Analysis
# Project Author: Ali Ashraf
# Project Date: 2021-12-12
# ---------------------------------------------------------------
# Description:
# ---------------------------------------------------------------
# Analyzing Uber data of 2014 and Explore and extract Uber boundaries and analyze the data of many different Df Manhattan, Bronx, Queens, and Staten Island and Brooklyn
# ---------------------------------------------------------------
```

Introduction:

Having that the trend of requiring via PM is becoming more common, this project will breakdown the options for Uber pick-ups and drop-offs in New York City during the year of 2014. The goal of this project is to provide insight to the average number of trips for establishments in these area locations. The original dataset used had many different Uber pick-up and drop-off locations in New York City, but it was too large to work with and there was a large amount of blank data. Leads to the conclusion that the data of Bronx and Manhattan has the greatest number of pick-ups and drop-offs. Staten Island has the least number of pick-ups and drop-offs. Brooklyn and Queens are relatively close to the Manhattan's. Bronx is the 2nd establishments in these areas, as it seems has many connections Uber will provide.

```
import pandas as pd
df = pd.read_csv('uber.csv')
df.to_csv('pickup_dropoff_Extracted_Extremes.csv')
df.to_csv('pickup_dropoff_Extracted_Extremes.csv', index=False)
df.head()
```

DataFrame:

	Unnamed: 0	key	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count
0	24238194	2015-05-07 19:52:06.0000003	7.5	2015-05-07 19:52:06 UTC	-73.999817	40.738354	-73.999512	40.723217	1.0
1	27835199	2009-07-17 20:04:56.0000002	7.7	2009-07-17 20:04:56 UTC	-73.994355	40.728225	-73.994710	40.750325	1.0
2	44984355	2009-08-24 21:45:00.0000061	12.9	2009-08-24 21:45:00 UTC	-74.005043	40.740770	-73.962565	40.772647	1.0
3	25894730	2009-06-26 08:22:21.0000001	5.3	2009-06-26 08:22:21 UTC	-73.976124	40.790844	-73.965316	40.799070	3.0
4	17610152	2014-08-28 17:47:00.000000188	16.0	2014-08-28 17:47:00 UTC	-73.925023	40.744085	-73.973082	40.761247	5.0

DataFrame info:

```
df.info()
df.describe()
df.shape
df.isnull().sum
df.isnull().sum()/df.shape[0]*100
```

```
RangeIndex: 200000 entries, 0 to 199999
Data columns (total 9 columns):
 Unnamed: 0         200000 non-null int64
 key                200000 non-null object
 fare_amount        200000 non-null float64
 pickup_datetime    200000 non-null object
 pickup_longitude   200000 non-null float64
 pickup_latitude    200000 non-null float64
 dropoff_longitude  199999 non-null float64
 dropoff_latitude   199999 non-null float64
 passenger_count    200000 non-null int64
dtypes: float64(5), int64(3), object(1)
memory usage: 13.7+ MB
```

	Unnamed: 0	fare_amount	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count
count	2.000000e+05	200000.000000	200000.000000	200000.000000	199999.000000	199999.000000	200000.000000
mean	2.771250e+07	11.359955	-72.527638	39.935885	-72.525292	39.923890	1.684535
std	1.601382e+07	9.901776	11.437787	7.720539	13.654671	6.795944	1.385997
min	1.000000e+00	-52.00000	-1340.648410	-74.015515	-3356.666300	-881.985513	0.000000
25%	1.382535e+07	6.000000	-73.992065	40.734796	-73.991407	40.733823	1.000000
50%	2.774687e+07	8.500000	-73.981823	40.752592	-73.980093	40.753042	1.000000
75%	4.155530e+07	12.500000	-73.968326	40.767154	-73.963659	40.768001	2.000000
max	5.542357e+07	499.000000	57.418457	1644.421482	1153.572603	872.697628	208.000000

```
(200000, 9)
```

```
Unnamed: 0         0
key                0
fare_amount        0
pickup_datetime    0
pickup_longitude   0
pickup_latitude    0
dropoff_longitude  1
dropoff_latitude   1
passenger_count    0
dtype: int64
```

```
Unnamed: 0         0.000000
key                0.000000
fare_amount        0.000000
pickup_datetime    0.000000
pickup_longitude   0.000000
pickup_latitude    0.000000
dropoff_longitude  0.000005
dropoff_latitude   0.000005
passenger_count    0.000000
dtype: float64
```

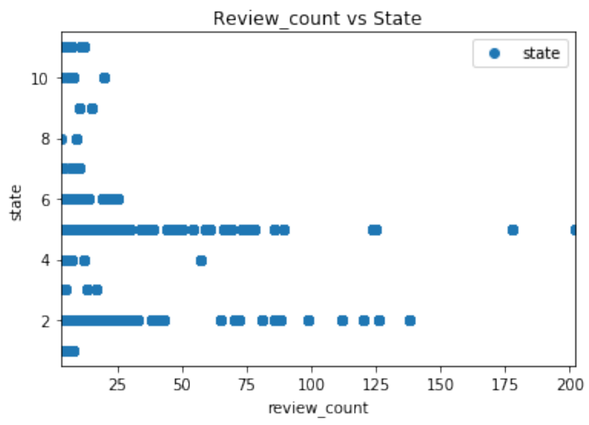
Data Analysis:

**Below we can see the dataset accessible for analytic purposes provided by Meg. In order to get the most out of this data set, it is best to drop the long factors to extract unnecessary data and have a cleaner dataframe to work with. Since we are looking at the pick-up and drop-off locations, we will drop the key, fare_amount, and the UTC CDR file. Once imported into the notebook I used the drop function to drop the unnecessary data, such as longitude and latitude. Since we are looking at the pick-up and drop-off locations, we will use the drop function to replace all the data with corresponding numbers in order to better examine the data, we used a factor function to do so. Once the data has been cleaned, we can now start to analyze the data. The new factors we realize as listed: 'c_lon' Rating', 'p_lon' Rating', 'c_lat' Rating', 'p_lat' Rating', 'p' Rating', 'p' Rating', 'c' Rating', 'c' Rating', 'c

	89032	8.0
	89044	125.0
	89074	44.0
	89081	5.0
	89102	46.0
	89103	44.0
	89104	14.0
	89106	3.0
5	89108	10.0
	89109	25.0
	89115	35.0
	89118	54.0
	89119	124.0
	89120	75.0
	89121	78.0
	89122	44.0
	89123	202.0
	89139	30.0
	89142	61.0
	89143	4.0
	89156	4.0
	89178	7.0
	89191	16.0
	44039	5.0
	44044	12.0
	44065	19.0
	44092	6.0
	44094	4.0
	44095	6.0
6	44122	22.0
	44146	6.0
	44203	4.0
	44212	7.0
	44241	25.0
	44255	12.0
	44256	7.0
	44264	14.0
	L0G 1V0	4.0
	L7E 1M1	3.0
7	M1B	7.0
	M1B 5W8	10.0
	M6H 2A2	8.0
8	15106	9.0
	15147	3.0
9	J0L 2K0	15.0
	J0N 1E0	10.0
	29708	20.0
10	29715	8.0
	29732	6.0
	53532	11.0
	53555	6.0
	53562	6.0
11	53589	4.0
	53597	5.0
	53716	5.0
	53718	5.0

Figure 1.9

In [230]:
```
yelp_dict2_df.plot(x='review_count', y='state', style='o')
plt.title('Review_count vs State')
plt.xlabel('review_count')
plt.ylabel('state')
plt.show()
```



Modeling Results:

Above in figure 1.7, we can see the use of an SNS pairgrid to visualize and model the data. I found it a bit challenging to get the plots and figures to show the visualization of data that I envisioned. We can see in SNS lmplot, figure 1.3 the spread of states and their related reviews. And the linear regresion model, figure 1.9, we can see the plotting of states and their related review counts. Being that there are two states that take up more than 50 percent of total data, I think this may have caused the tests to be flawed.

Conclusion:

After running these models and testing, I came to the conclusion that the states of Arizona and Nevada most likely have the highest number of reviews due to the fact that these states have a very tourist friendly monuments or are the closests states possible to start a cross-country road trip. I would recommend to consumers who are intersted in the RV category to choose a state in which there is a healthy number of tourism so they get the best our their RV needs.

In [ ]: